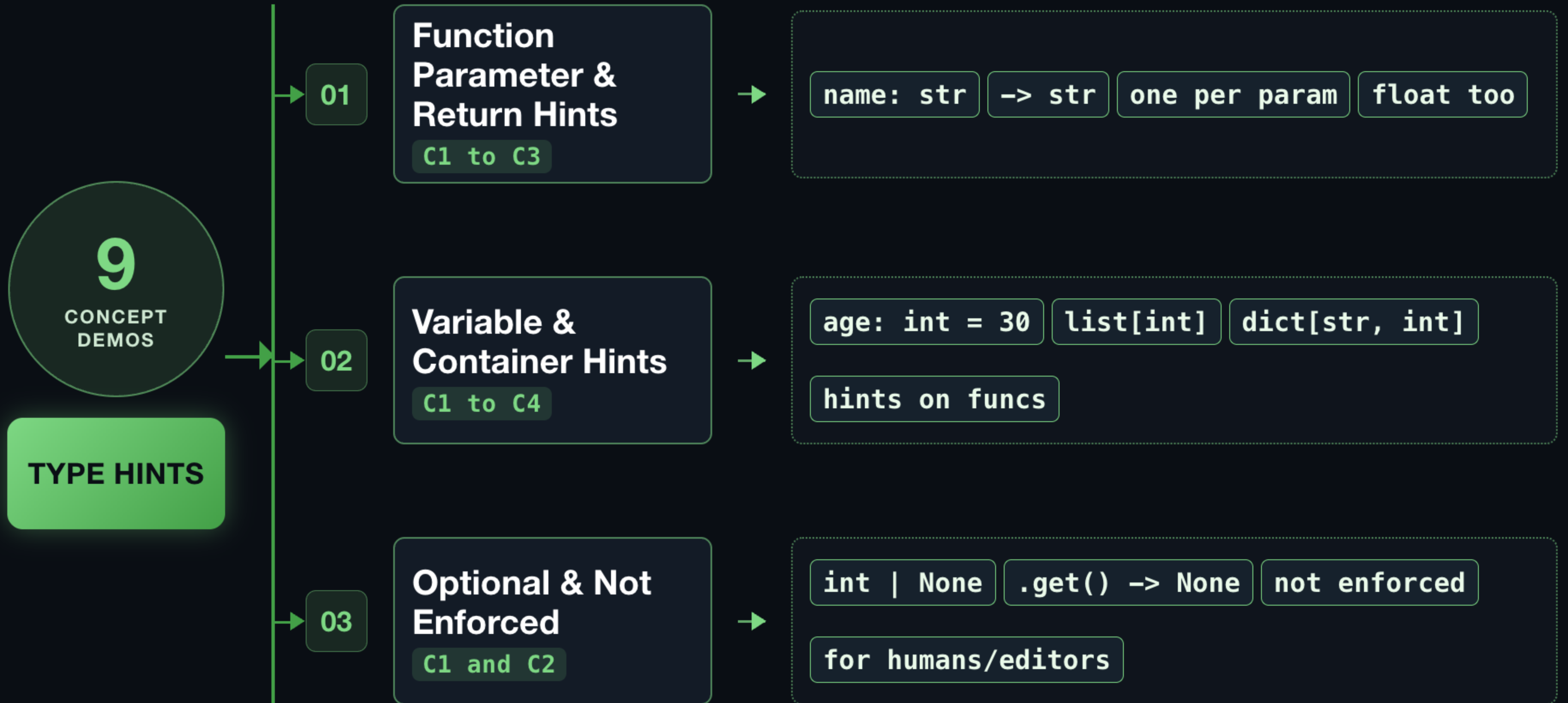


Section 08: Type Hints **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Type Hints: the colon and the arrow

A hint is a LABEL, not a rule

It tells you and your editor what a function **expects** and what it **gives back**. It does not change how the code runs.

THE SAME FUNCTION
WITHOUT HINTS

`def average(a, b):` Perfectly valid Python. It just never says what a and b should be, or what you get back.

`def average(a: float, b: float) -> float:`

after each colon, what goes IN

a: float and b: float label the two parameters

after the arrow, what comes OUT

-> float labels the returned value

THE HINTED DEF

`def greet(name: str) -> str:`

`def area(width: int, height: int) -> int:`

`def average(a: float, b: float) -> float:`

THE CALL

`greet("Kalyan")`

`area(4, 5)`

`average(10.0, 20.0)`

GIVES

Hello, Kalyan!

20

15.0

The shape never changes. Every parameter gets a type after its **colon**, and the whole function gets one type after the **arrow**.

Type Hints: Function Parameter and Return Hints

A **type hint** labels each parameter after a colon and the return value after `->`, so editors and readers know the types a function expects and gives back.

#	CONCEPT	SAMPLE CODE
1	Parameter and return hint <code>: str</code> <code>-></code> Colon marks the input type, <code>-></code> marks the return type.	<pre>def greet(name: str) -> str:</pre>
2	Several parameters <code>int</code> Each parameter carries its own type hint.	<pre>def area(width: int, height: int) -> int:</pre>
3	Decimal number hints <code>float</code> Use float when values can carry decimals.	<pre>def average(a: float, b: float) -> float:</pre>

Type Hints: Variables and Containers

A container hint has two halves

The word outside the brackets is the **container**; the part inside is **what it holds**. `list[int]` is not just "a list", it is a list of whole numbers.

A plain variable

name : type, one type, nothing inside

```
name: str = "Kalyan"
```

```
age: int = 30
```

```
price: float = 19.99
```

```
is_active: bool = True
```

```
PRINTS           Kalyan 30 19.99 True
```

A container

the brackets say what is held inside

```
scores: list[int] = [70, 95, 60, 88]
```

```
stock: dict[str, int] = {"pens": 3, "books": 1}
```

In `dict[str, int]` the first type is the key, the second is the value, so this maps text keys to whole numbers.

```
sum(scores)           313
```

```
stock["pens"]         3
```

It works the same on a function: `def total(prices: list[int]) -> int:` takes a list of numbers and gives one number back. `total([10, 20, 30])` is 60.

Type Hints: Variable and Container Hints

A type hint labels a variable with its type, and a **container hint** like `list[int]` says exactly what type sits inside.

#	CONCEPT	SAMPLE CODE
1	Variable hint <code>: type</code> Put a colon and the type after the name.	<pre>name: str = "Kalyan"</pre>
2	List hint <code>list[int]</code> Say a list holds whole numbers inside.	<pre>scores: list[int] = [70, 95, 60, 88]</pre>
3	Dict hint <code>dict[str, int]</code> Map text keys to whole-number values.	<pre>stock: dict[str, int] = {"pens": 3}</pre>
4	Function hint <code>-> int</code> Hint the input type and the return type.	<pre>def total(prices: list[int]) -> int:</pre>

Type Hints: what a hint **promises**, and what it does **not**

A hint is a message to the reader, not a guard on the door

It can **tell** you that a value might be missing. It cannot stop anyone passing the wrong thing. Nothing does, until you reach for a tool that checks.

What it **DOES** tell you

"or None" tells you the answer may be missing, so you handle it

```
def find_price(name: str) -> int | None:
```

```
find_price("pen")      2
```

```
find_price("eraser")  None
```

The hint told you None was possible

so you check before you use the value

What it does **NOT** do

Python does not check hints while the code runs

```
def greet(name: str) -> str:
```

```
greet("Kalyan")  Hello, Kalyan!
```

```
greet(123)       Hello, 123!
```

A number was passed and it still ran

Python ran it exactly as written

So why bother? Because other tools **read what Python ignores**. The next slide shows exactly who reads a hint, and which of them will actually stop bad data.

Type Hints: who actually enforces them

Python never checks a hint. Other tools choose to.

The hint `name: str` is the same text in every row below. What changes is who reads it, and when.

WHO READS THE HINT	WHEN	ENFORCES?	WHAT YOU ACTUALLY SEE
Python itself	while it runs	NO	<code>greet(123)</code> runs and prints <code>Hello, 123!</code>
<code>@dataclass</code>	while it runs	NO	without type hints there is no <code>__init__</code> constructor; even then, the values are never checked
Pydantic	while it runs	YES	checks every value against its hint, and converts what it safely can
FastAPI	on every request	YES	bad input never reaches your function, because Pydantic checks it first

IS IT A BEST PRACTICE? Yes. They cost nothing while your code runs, they tell the next reader what you meant, and **Pydantic** and **FastAPI** cannot check anything without them.

So why write hints at all? Python ignores them, but the tools built on top of Python do not. You write the hint **once**, and **Pydantic** and **FastAPI** read it and put it to work.

Type Hints: Optional Values and Not Enforced

The **X | None** hint marks a value that might be missing, and Python never enforces hints when the code runs; they guide people and tools only.

#	CONCEPT	SAMPLE CODE
1	Optional value <code>X None</code> Says a value is that type, or None when missing.	<pre>def find_price(name: str) -> int None:</pre>
2	Hints not enforced Python runs the code even when a hint is wrong.	<pre>print(greet(123)) # still runs</pre>